

Required External SXCP & SECTP Infrastructure

I. Ethereum & EVM-compatible External Chains

These are the on-chain contracts needed to make the chain SXCP-capable (and therefore usable from Synergy Wallet for cross-chain coordination):

1. **SXCPVault.sol** (chain-side vault/escrow + timeout/revert handling; emits the core SXCP events)
2. **WitnessRegistry.sol** (relayer registration, quorum config, reputation, slashing, audit trail hooks)
3. **SignatureVerifier.sol** (verifies ML-DSA aggregate/partials; loads keys from registry)
4. **GovernanceModule.sol** (parameter changes, emergency pause, relayer set changes)
5. **AuditLogger.sol** (attestation/slash/governance logging; history queries; storage strategy)
6. **StateProofValidator.sol** (Merkle/receipt/log proof validation)
7. **FinalityChecker.sol** (finality/confirmations/reorg detection)
8. **CompensationPool.sol** (fee intake + compensation allocation)
9. **RelayerRegistry.sol** (stake + rewards plumbing)

SECTP note (same-chain transfers): SECTP does **not** require deploying anything on external chains; it's explicitly "resolve by identity, execute natively" (local signing + direct broadcast). Synergy may optionally record an **audit receipt** on Synergy, but that's not an external-chain deployment requirement.

II. Solana & SVM-Compatible Chains

Solana uses **programs** (not Solidity contracts). Deploy the program equivalents of SXCP responsibilities:

1. **sxcp_vault_program** (chain-side vault/escrow + timeout/revert handling; emits the core SXCP events)
2. **witness_registry_program** (relayer registration, quorum config, reputation, slashing, audit trail hooks)
3. **signature_verifier_program** (verifies ML-DSA aggregate/partials; loads keys from registry)
4. **governance_program** (parameter changes, emergency pause, relayer set changes)
5. **audit_logger_program** (attestation/slash/governance logging; history queries; storage strategy)
6. **state_proof_validator_program** (Merkle/receipt/log proof validation)
7. **finality_checker_program** (finality/confirmations/reorg detection)
8. **comp_pool_program** (fee intake + compensation allocation)
9. **relayer_registry_program** (stake + rewards plumbing)

SECTP on Solana: no extra programs required—SECTP is native Solana transfers after identity→address resolution and local policy checks/signing.

III. Cosmos SDK external chains

Option A — *CosmWasm contracts*

1. **sxcp_vault_wasm**
2. **witness_registry_wasm**
3. **signature_verifier_wasm**
4. **governance_wasm**
5. **audit_logger_wasm**
6. **state_proof_validator_wasm**
7. **finality_checker_wasm**
8. **comp_pool_wasm**
9. **relayer_registry_wasm**

Option B — *native Cosmos module*

- *x/sxcp* module that implements the same roles as (vault, witness registry, verification, governance, audit, optional proof/finality/comp pool).

SECTP on Cosmos: no chain-side deployments required for basic same-chain sends; it's native execution after identity routing resolution.

IV. Substrate / Polkadot-parachain external chains

Deploy runtime pallets:

1. **pallet-sxcp-vault**
2. **pallet-sxcp-witness-registry**
3. **pallet-sxcp-signature-verifier**
4. **pallet-sxcp-governance**
5. **pallet-sxcp-audit**
6. **pallet-sxcp-proof-validator**
7. **pallet-sxcp-finality**
8. **pallet-sxcp-comp-pool**
9. **pallet-sxcp-relayer-registry**

SCETP on Substrate: no special on-chain deployment required for same-chain sends (native execution).

V. Bitcoin / UTXO external chains

Bitcoin doesn't do "deploy contracts" the same way. To support SXCP, you standardize **on-chain script templates + transaction forms** aligned with the **SXCP Atomic Swap Mode (HTLC-style)** described in the SXCP spec:

1. **HTLC locking script template** (hash-lock + time-lock; claim vs refund paths)
2. **Claim transaction template** (spends HTLC with preimage)
3. **Refund transaction template** (spends HTLC after timeout)
4. **Indexable intent marker patterns** (e.g., OP_RETURN conventions) explicit "intent declared" signals for relayers to watch

SCETP on Bitcoin: no extra artifacts—SCETP is native Bitcoin transactions after identity/address resolution and local signing/broadcast.

VI. Move VM chains (Aptos, Sui, Movement, etc.)

Form factor: Move **packages/modules** published on-chain (not EVM contracts):

1. **sxcp_vault.move** (escrow/lock + release + timeout paths; emits events)
2. **witness_registry.move** (relayer set, thresholds, reputation/slashing, attestation records)
3. **signature_verifier.move** (attestation verification endpoint; might be split as a library)
4. **audit_logger.move**
5. **governance.move**
6. **state_proof_validator.move,**
7. **finality_checker.move,**
8. **comp_pool.move**

Notes: Move has strong resource semantics; "vault" becomes a resource container + event emitter, which is a nice fit.

VII. Cairo / Starknet-style zkVM chains (Starknet and Cairo-based L2s)

Form factor: Cairo contracts (Sierra/Casm), with different event/log patterns:

1. ***sxcp_vault.cairo***
2. ***witness_registry.cairo***
3. ***signature_verifier.cairo***
4. ***audit_logger.cairo***
5. ***governance.cairo***
6. ***proof_validator.cairo***
7. ***finality_checker.cairo***
8. ***comp_pool.cairo***

Notes: If you rely on on-chain proof validation here, you'll need Cairo-friendly proof primitives; otherwise keep proof building off-chain and verify attestations on-chain.

VIII. WASM smart contract chains that are not Cosmos (NEAR, some L1s/L2s)

Form factor: WASM contracts (Rust/AssemblyScript), but not CosmWasm:

1. ***sxcp_vault.wasm***
2. ***witness_registry.wasm***
3. ***signature_verifier.wasm***
4. ***audit_logger.wasm***
5. ***governance.wasm***
6. ***proof_validator.wasm***
7. ***finality_checker.wasm***
8. ***comp_pool.wasm***

Notes: Similar roles as your CosmWasm option, but with chain-specific APIs/storage/event formats.

IX. Cardano / eUTXO smart contract chains

*Form factor: UTXO-like with **validator scripts** (Plutus) + **datum/redeemer** models. Not “deploy contracts” like EVM:*

1. **Validator script(s)** for *sxcp_vault* (locks, unlock conditions, timeouts)
2. **Minting policy script(s)** if you represent assets via mint/burn semantics
3. **Standardized datum/redeemer schemas** (the “message format” of the contract)
4. **Transaction construction templates** (claim/refund paths, attestation inclusion)

Notes: Operationally this resembles Bitcoin/UTXO, but with richer scripting and state carried in UTXO data.

X. Algorand / AVM chains

Form factor: *TEAL programs + stateful smart contracts (Application IDs) + LogicSig:*

1. ***sxcp_vault_app*** (stateful application for escrow/lock/release/timeout)
2. ***witness_registry_app***
3. ***signature_verifier_app*** (or library TEAL; often verification is constrained)
4. ***audit_logger_app***
5. ***governance_app***
6. ***proof/finality/comp pool apps if feasible***

Notes: May choose “attestation-only on-chain verification” if PQ verification isn’t practical on AVM.

XI. Tezos / Michelson-family chains

Form factor: *Michelson contracts (often authored in LIGO or SmartPy):*

1. ***sxcp_vault.tz***
2. ***witness_registry.tz***
3. ***signature_verifier.tz***
4. ***audit_logger.tz***
5. ***governance.tz***
6. ***proof_validator.tz***
7. ***finality_checker.tz***
8. ***comp_pool.tz***

Notes: Events/logs differ; indexing is typically via operation results/storage updates.

XII. XRPL / Stellar-style “limited VM” payment networks

These vary a lot.

Stellar (Soroban)

Form factor: *Soroban contracts (Rust/WASM).*

Deploy the usual set:

- *sxcp_vault, witness_registry, signature_verifier, audit_logger, governance, optional extras.*

XRPL

Form factor: *Historically limited; newer “Hooks” exist in some environments.*

- *If **Hooks** are available: implement a minimal vault hook + registry hook pattern.*
- *If not: may be limited to **standardized transaction forms + memo conventions** (closer to UTXO template style).*

Notes: *For networks with limited programmability, SXCP support may look more like “templates + indexable markers” than full on-chain modules.*

XIII. TON (The Open Network)

Form factor: Smart contracts in **Func/Tact** with message-passing semantics.

1. ***sxcp_vault contract*** (handles internal/external messages, timeouts)
2. ***witness_registry***
3. ***signature_verifier***
4. ***audit_logger***
5. ***governance***
6. ***proof/finality/comp pool***

Notes: TON's async message model means "vault lifecycle" must be explicitly stateful with clear bounce/timeout handling.